DATA ENGINEERING SDLC & DATA CONTRACTS

Production Reference: Open Data Contract Standard (ODCS v3), Semantic Versioning & Shift-Left Testing Pyramid

ODCS V3.0 | SHIFT-LEFT TESTING | DATA CONTRACTS | ISO 216 A4

Open Data Contract Standard (ODCS v3)

Interface Design

```yaml
# orders_contract.yaml - Open Data Contract Standard v3
version: "3.0.0"
dataset: "prod_lakehouse.silver_ecommerce.orders"
owner: "checkout-team@enterprise.com"
schema:
  - name: order_id
    type: string
    required: true
    unique: true
  - name: total_amount
    type: decimal(18,2)
    required: true
    logical_bounds: { min: 0.01, max: 1000000.00 }
  - name: order_status
    type: string
    enum: ["PENDING", "AUTHORIZED", "SHIPPED", "REFUNDED"]
sla:
  freshness: "15m"        # Maximum allowed pipeline delay
  expected_daily_volume: { min: 50000, max: 2000000 }
```

CONTRACT PRIMITIVES

- **Producer Responsibility:** Upstream services validate against contract prior to landing payloads in Bronze Kafka/S3.
- **Automated Tooling:** Parsed by CI/CD to generate JSONSchema, Great Expectations suites, and dbt schema yaml.

Semantic Versioning for Data Assets

Governance

```
-- Dataset SemVer Guidelines (MAJOR.MINOR.PATCH)
1. MAJOR Version Bump (v1.0.0 → v2.0.0) [BREAKING]:
   - Column removal or column rename.
   - Type narrowing (e.g. DOUBLE → INT, or widening that breaks target systems).
   - Making previously nullable column NOT NULL.
   - Removing enum members or changing business definition of metric.
   Protocol: Dual-write to orders_v1 and orders_v2 for 90-day deprecation grace.

2. MINOR Version Bump (v1.1.0 → v1.2.0) [BACKWARD COMPATIBLE]:
   - Adding new nullable column with default value.
   - Adding new valid enum members (if consumers handle open enums).
   - Relaxing constraint (e.g. NOT NULL → nullable).

3. PATCH Version Bump (v1.1.1 → v1.1.2) [METADATA ONLY]:
   - Documentation, description, or data dictionary updates.
```

MIGRATION GOVERNANCE

- **Zero Breaking In-Place:** Breaking changes in production without dual-write violates data contracts and triggers P1 outage.
- **Deprecation Alerting:** Monitor audit logs to verify all consumers migrated before dropping `v1` endpoints.

Shift-Left Testing Pyramid for Lakehouse

Quality Engineering

```
-- 4-Tier Lakehouse Testing Architecture
┌──────────────────────────────────────────────────────────┐
│ Tier 4: Reconciliation & Anomaly Detection               │
│ Production monitoring: Volume drift, distribution shift  │
├──────────────────────────────────────────────────────────┤
│ Tier 3: Medallion Integration Tests                      │
│ Silver/Gold table assertions: End-to-end ACID pipeline   │
├──────────────────────────────────────────────────────────┤
│ Tier 2: Ingress Contract Verification                    │
│ Schema validation, null checks, enum conformance (DLQ)   │
├──────────────────────────────────────────────────────────┤
│ Tier 1: Fast Local Unit Tests (80% of test suite)        │
│ Pure business logic functions tested with PyTest/Chispa  │
└──────────────────────────────────────────────────────────┘
```

TESTING PYRAMID DISTRIBUTION

- **Unit Testing (PyTest + Chispa):** Runs locally in <15 seconds using mock DataFrames without spinning up cloud clusters.
- **Fast Feedback:** 95% of logic bugs caught at Tier 1 before pushing git commit to remote branch.

Dead Letter Queue (DLQ) & Quarantine

Fault Tolerance

```
-- PySpark Streaming Schema Quarantine & DLQ Flow
valid_records = raw_df.filter(
    col("order_id").isNotNull() &
    (col("total_amount") > 0) &
    col("order_status").isin("PENDING", "AUTHORIZED", "SHIPPED")
)

quarantined_records = raw_df.filter(
    col("order_id").isNull() |
    (col("total_amount") ⩽ 0) |
    ~col("order_status").isin("PENDING", "AUTHORIZED", "SHIPPED")
).withColumn("rejection_reason", lit("FAILED_CONTRACT_RULES"))  .withColumn("rejected_at", current_timestamp())

-- Divert invalid payloads to Bronze quarantine table; pipeline never crashes!
quarantined_records.write.format("delta").mode("append").saveAsTable("bronze.orders_dlq")
```

QUARANTINE PRINCIPLES

- **Non-Stop Pipeline:** Corrupt source records are quarantined without blocking processing of valid downstream rows.
- **Automated PagerDuty:** If `dlq_rate > 1.5%` over 10-minute window, trigger emergency alert to source upstream.

Pre-Commit & Local Developer Tooling

Developer Velocity

```yaml
# .pre-commit-config.yaml - Enforce quality locally before git push
repos:
  - repo: https://github.com/sqlfluff/sqlfluff
    rev: 3.0.0
    hooks:
      - id: sqlfluff-lint
        args: ["--dialect", "databricks"]
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.0
    hooks:
      - id: ruff
        args: ["--fix"]
  - repo: local
    hooks:
      - id: pytest-unit
        name: PyTest Fast Logic Unit Tests
        entry: pytest tests/unit -m "not slow"
        language: system
        pass_filenames: false
```

DIALECT RULE ENFORCEMENT

- **SQLFluff Dialect:** Enforces lowercase keywords, comma placement, explicit column aliasing, and qualified table references.
- **Ruff Speed:** Executes Python linting and code formatting in <50ms across entire repository.

Contract-Driven Ingestion Verification

Automated Verification

```
# Python Pydantic / Pandera Ingestion Gate
from pandera.pyspark import DataFrameModel, Field
from pyspark.sql.types import StringType, DecimalType

class OrdersContractModel(DataFrameModel):
    order_id: StringType = Field(nullable=False, unique=True)
    customer_id: StringType = Field(nullable=False)
    total_amount: DecimalType = Field(ge=0.01, le=1000000.0)
    order_status: StringType = Field(isin=["PENDING", "AUTHORIZED", "SHIPPED"])

# Validate Ingested DataFrame:
try:
    validated_df = OrdersContractModel.validate(raw_df)
except SchemaError as err:
    logger.error(f"Contract breached: {err}")
    send_slack_alert(channel="#data-ops", msg=str(err))
```

SCHEMA VERIFICATION BENEFITS

- **Type Strictness:** Catches unexpected string floats, null IDs, and unmapped status codes before entering Silver tier.
- **Living Documentation:** Pydantic/Pandera models serve as executable code contracts checked into version control.

CI/CD GATES, BLUE/GREEN & INCIDENT RUNBOOK

Production Reference: Ephemeral Slim CI, Zero-Downtime Blue/Green Swaps & SLA Breach Emergency Protocol

CI/CD GATES | BLUE/GREEN DEPLOY | SLIM CI | ISO 216 A4

Automated Ephemeral Slim CI Gates

CI/CD Pipeline

```yaml
# GitHub Actions: dbt Slim CI against Ephemeral Schema
name: Pull Request Data Pipeline Gate
on: [pull_request]
jobs:
  slim_ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Download Production Manifest
        run: aws s3 cp s3://prod-manifests/manifest.json ./prod-manifest.json
      - name: Run Modified Models & First-Degree Downstream
        run: |
          dbt build --target ci            --select "state:modified+"            --de
fer --state ./            --vars 'schema_suffix: pr_${{ github.event.number }}'
      - name: Teardown Ephemeral Schema on Merge/Close
        if: always()
        run: dbt run-operation drop_pr_schema --args '{"pr_id": "${{ github.event.numbe
r }}"}'
```

SLIM CI MECHANICS

- **State Deferral:** Unmodified upstream models read production tables directly without recomputing historical data.
- **Cost Elimination:** PR validation finishes in <3 minutes while verifying exact column dependencies.

Zero-Downtime Blue/Green Table Deployment

Deployment

```sql
-- 1. Create Staging Clone (Zero data copy via Delta Shallow Clone)
CREATE TABLE gold_orders_green SHALLOW CLONE gold_orders_blue;

-- 2. Execute High-Impact Transform & Backfill on Green Table
INSERT INTO gold_orders_green SELECT ... FROM silver_orders_v2;

-- 3. Execute Automated Acceptance Validation Suite on Green:
ASSERT (SELECT count(*) FROM gold_orders_green) > 100000;
ASSERT (SELECT count(*) FROM gold_orders_green WHERE id IS NULL) = 0;

-- 4. Atomic View Pointer Swap (Zero downtime, sub-millisecond switch!)
CREATE OR REPLACE VIEW gold.orders AS SELECT * FROM gold_orders_green;

-- 5. Instant Rollback Capability (if metric anomaly surfaces):
CREATE OR REPLACE VIEW gold.orders AS SELECT * FROM gold_orders_blue;
```

DEPLOYMENT SAFEGUARDS

- **Consumer Isolation:** BI dashboards and ML inference services query stable `gold.orders` view; zero connection drops.
- **Blue Retention:** Retain blue table for 48 hours before dropping storage to permit instantaneous rollback.

Automated Schema Drift Detection

Observability

```python
-- Delta Lake Auto-Merge Schema Evolution vs Drift Trap
-- SAFE: Controlled additive column evolution:
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "false")
-- Keep autoMerge DISABLED in production to prevent silent corruption!

-- Explicit Drift Trap Detection Query:
incoming_cols = set(incoming_df.columns)
contract_cols = set(contract.get_columns())

if not incoming_cols.issubset(contract_cols):
    unexpected_cols = incoming_cols - contract_cols
    logger.warn(f"UNAUTHORIZED SCHEMA DRIFT: {unexpected_cols}")
    divert_to_unmapped_payload(incoming_df)
elif missing := contract_cols - incoming_cols:
    raise ContractViolationException(f"Missing required contract columns: {missing}")
```

DRIFT PREVENTION RULES

- **AutoMerge Hazard:** Enabling `autoMerge=true` indiscriminately turns typos like `cusotmer_id` into permanent columns.
- **Alert Webhooks:** Transmit schema drift anomalies directly to data platform Slack channel with author attribution.

Data Product Lifecycle & Deprecation

Product Management

```
-- 4-Stage Deprecation Protocol
Phase 1: Announcement (T - 90 Days)
  - Publish contract notice in Data Catalog (Unity Catalog / Purview).
  - Add tag lifecycle: "deprecated" and sunset_date: "2027-02-15".
Phase 2: Active Dual-Write (T - 60 Days)
  - Ingestion pipeline mirrors writes to both v1 and v2 tables.
Phase 3: Brownout Testing (T - 14 Days)
  - Temporarily disable v1 access for 4 hours during non-peak business window.
  - Surfaces undocumented or silent consumer pipelines before hard cutoff!
Phase 4: Sunset & Archival (T = 0)
  - Revoke read permissions on v1, snapshot final state, archive to cold tier.
```

BROWNOUT BEST PRACTICE

- **The Brownout Test:** The only 100% reliable method to find legacy cron jobs and rogue spreadsheets querying deprecated assets.
- **Contact Registration:** Maintain consumer owner metadata inside catalog for automated notifications.

Production Contract Breach Protocol

Emergency Ops

```
-- Emergency Triage Sequence for Broken Data SLAs / Schemas
Step 1: Isolate & Divert
  - Halt automated ingestion to Silver tables immediately:
    ALTER PIPELINE orders_pipeline PAUSE;
  - Divert raw incoming Kafka records to dead letter storage bucket.
Step 2: Stakeholder Notification
  - Post P1 incident ticket to status page: "Silver Orders Degraded".
Step 3: Root-Cause Classification
  - Check whether failure is: (A) Source payload mutation, (B) Compute cluster OOM, or
(C) Upstream database outage.
Step 4: Remediation & Backfill
  - Patch contract validation rules, run idempotent backfill:
    REPLACE INTO silver_orders SELECT ... FROM raw_quarantine;
```

INCIDENT OWNERSHIP

- **Producer Responsibility:** When source teams mutate schemas unannounced, source team owns MTTR resolution.
- **SLA Impact:** Document exact downstream dashboard impact and downtime minutes in post-mortem.

FinOps & Lakehouse CI/CD Cost Controls

Cost Engineering

```
-- Lakehouse CI/CD Resource Rightsizing Rules
1. Auto-Termination for Test Clusters:
   - Enforce autotermination_minutes: 10 on all PR evaluation clusters.
2. Ephemeral Serverless SQL Warehouses:
   - Use Serverless 2X-Small warehouses for dbt testing; scales down in 5s!
3. Partition Pruning in CI Suites:
   - Restrict integration testing datasets to last 3 days:
   dbt test --vars 'test_start_date: 2026-11-10'
4. Cache Layer Reuse:
   - Cache Python virtual environments, dbt packages, and dependencies in CI.
5. GitHub Actions Concurrency Cancellation:
   - Cancel outdated PR builds when new commits are pushed to same branch.
```

FINOPS INVARIANTS

- **Cost Savings:** Slim CI + 10-minute auto-termination cuts monthly data engineering compute spend by 65%.
- **Budget Alerts:** Set daily workspace spend anomaly detection alerts at $200 variance.